Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

Smart Campus NOC Simulator 2.0

24BAI0159 ANUSHREE BHARGAVA

24BCI0180 PRANJAL SAXENA

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



October 2026

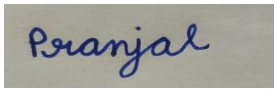
ABSTRACT

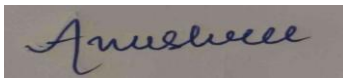
This report presents Smart Campus Network, an interactive software-based platform for designing, simulating and evaluating a secure, fault-tolerant campus computer network. The system combines a React and TypeScript dashboard with a Node.js/Express backend, an sql.js (SQLite) persistence layer and dedicated simulation engines. The model covers a redundant three-router core, nine departmental VLANs, IPv4/IPv6 addressing and VLSM tools, OSPF-style (Dijkstra) and RIP-style (distance-vector) routing, first-match firewall access-control lists (ACLs), multi-hop packet tracing, strict-priority quality-of-service (QoS) queueing against FIFO tail-drop, link-failure injection, and educational models of DHCP, DNS, HTTP, FTP, NAT/PAT and data-link protocols. Nine standalone C programs implement the syllabus algorithms. In the fault-tolerance experiment, severing the R1–R2 link rerouted traffic through R3 in both cases, but OSPF-style routing converged in 46 ms with 1% transient loss, whereas RIP-style routing needed 3,557 ms and lost 53% of the test packets. The QoS experiment delivered all 2,000 generated packets under both queueing policies with 0% voice loss. ACL tests confirmed that student and IoT traffic to the Administration VLAN is denied. The implementation passed 23 automated tests and 22 API integration checks, and both production builds succeeded. Results are simulator outputs from configured models and synthetic traffic rather than physical-network measurements; future work includes repeated-run statistics and validation on an emulated testbed.

Keywords: Smart campus network, OSPF, RIP, VLAN, ACL, QoS, fault tolerance, packet tracing, network simulation.

Certificate

This is to certify that the project report titled “Design and Performance Analysis of a Secure, Fault-Tolerant Smart Campus Network” is a bonafide record of the work done by Anushree Bhargava (24BAI0159) and Pranjal Saxena (24BCI0180) in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Pranjal Saxena 

Anushree Bhargava 

Name and Signature of the Students

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**2**
1.	**INTRODUCTION**	**6**
	1.1 Background and motivation	6
	1.2 Problem statement	6
	1.3 Objectives	6
	1.4 Scope and limitations	6
	1.5 Mapping to course modules and the lab experiment	7
2.	**LITERATURE SURVEY**	**7**
	2.1 Overview of the existing protocol or scheme	7
	2.2 Review of related work	7
	2.3 Research gap and justification of novelty	8
3.	**PROPOSED SYSTEM**	**8**
	3.1 System architecture or topology	8
	3.2 Baseline mechanism description	9
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	9
	3.4 Novelty statement	10
	3.5 Assumptions and design decisions	10
4.	**IMPLEMENTATION**	**10**
	4.1 Tools, platform, versions and hardware	10
	4.2 Simulation or testbed setup	11
	4.3 Code structure and explanation of key modules	11
	4.4 Challenges faced and how they were solved	12
5.	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	**13**
	5.1 Performance metrics and why they were chosen	13
	5.2 Scenarios and parameter values	13
	5.3 Results: tables and graphs, baseline vs proposed	13
	5.4 Analysis and discussion	15

5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails 15

6. CONCLUSION AND FUTURE WORK 16

6.1 Summary of work and key findings 16

6.2 Limitations 16

6.3 Future enhancements 16

7. REFERENCES 17

References (IEEE style) 17

Appendix A: Important source code or configuration listings 18

Appendix B: Additional screenshots, packet captures, raw result tables 20

1. INTRODUCTION

1.1 Background and motivation

A campus network connects academic departments, student devices, faculty offices, administrative systems, servers, libraries, and IoT equipment. These groups have different access needs and traffic patterns. A useful design must provide connectivity while limiting unauthorised access, responding to link failures, and handling congestion. In a physical lab, demonstrating all such situations can require multiple routers, switches, endpoints, and configuration time.

Smart Campus Network addresses this teaching need through a browser-based network model. Students can inspect a logical topology, change simulation inputs, follow packet forwarding decisions, compare routing behaviours, and run controlled failure or congestion scenarios. It is a simulator and learning platform, not a claim that a production campus network has been deployed.

1.2 Problem statement

The main problem is how to demonstrate, in one repeatable lab environment, the interaction between network addressing, routing, redundancy, access control, traffic prioritisation, and common services. Static diagrams alone do not show how a route changes after a link failure or why an ACL denies a packet. The project provides interactive models to make these decisions visible.

1.3 Objectives

- Model a campus topology with a redundant routing core and multiple departmental VLANs.
- Provide IPv4 subnet calculation, VLSM allocation, IPv6 prefix formatting, and subnet membership checks.
- Compare OSPF-style shortest-path routing with RIP-style distance-vector routing.
- Demonstrate alternate-path routing after a simulated link failure and show the no-alternate-path case.
- Evaluate ordered ACL rules and explain allow/deny decisions for packet fields.
- Compare strict-priority queueing with FIFO tail-drop under a simulated bottleneck.
- Simulate common network services and data-link/networking algorithms through an interactive interface.
- Provide automated tests, guided demonstrations, and experiment reports suitable for a lab viva.

1.4 Scope and limitations

The scope includes software simulations, dashboard visualisation, stored topology data, and standalone C programs for selected networking algorithms. The model does not emulate every timer, vendor-specific feature, physical-layer effect, or full standards-compliant protocol state

machine. Traffic, packet loss, and convergence are generated by application logic. Results must be interpreted as simulator outputs, not field measurements.

1.5 Mapping to course modules and the lab experiment

Table 1. Mapping of course concepts to the project implementation

Computer Networks concept	Project implementation	Learning outcome
IP addressing and subnetting	IPv4 subnet calculator, VLSM, IPv6 prefix formatting	Calculate network boundaries and usable address ranges
Routing and forwarding	Dijkstra/OSPF-style and Bellman–Ford/RIP-style routing	Compare path selection and routing-table behaviour
Network layer and packet delivery	Multi-hop packet tracer	Observe per-hop forwarding and packet drops
Network security	First-match ACL engine and VLAN separation	Explain policy-based allow/deny decisions
Congestion and QoS	Priority queues vs FIFO tail-drop	Study delay, throughput, and packet-drop trade-offs
Reliable data transfer/error control	Hamming, CRC, checksum, Go-Back-N, Selective Repeat	Understand error detection/correction and retransmission
Application/network services	DHCP, DNS, HTTP, FTP, NAT/PAT simulations	Trace basic service requests and address translation
Medium access control	ALOHA, slotted ALOHA, CSMA and CSMA/CD C programs	Compare contention-based channel access methods

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

The project uses established networking concepts as baselines. OSPF is a link-state routing protocol; shortest-path calculations are commonly explained using Dijkstra's algorithm. RIP is a distance-vector protocol that selects routes using hop count and has a maximum usable distance of 15 hops. VLANs separate broadcast domains logically. ACLs apply ordered rules to packet attributes. FIFO queueing treats packets in arrival order, whereas strict-priority queueing serves higher-priority traffic before lower-priority traffic. The project models these concepts for learning rather than implementing complete production protocol stacks. The baselines are drawn from the standards listed in Section 2.2 [1]–[6].

2.2 Review of related work

The following references describe the standards and algorithms directly relevant to the implementation. The table summarises these foundational standards, which define the baseline behaviour modelled in the project.

Table 2. Literature survey of the standards used as baselines

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Moy, 1998 [1]	OSPF Version 2, RFC 2328	Link-state routing and shortest-path computation	Defines interoperable OSPF behaviour	A standard, not an interactive comparison lab
2	Malkin, 1998 [2]	RIP Version 2, RFC 2453	Distance-vector routing and hop count	Documents RIP route exchange and limits	Does not provide this project's visual failover experiment
3	Deering & Hinden, 1998 [3]	Internet Protocol, Version 6, RFC 2460	IPv6 addressing and packet format	Foundational IPv6 specification	Not a teaching dashboard or VLSM planner
4	Baker, 1995 [4]	Requirements for IP Version 4 Routers, RFC 1812	Router forwarding and control behaviour	Clarifies router responsibilities	Does not compare simulated campus scenarios
5	Braden, 1989 [5]	Requirements for Internet Hosts, RFC 1122	Host communication requirements	Broad baseline for Internet networking	Not a UI-based simulator or performance study
6	Postel, 1980 [6]	User Datagram Protocol, RFC 768	Datagram service and ports	Simple transport-layer reference	Does not address queue policy or campus security design

2.3 Research gap and justification of novelty

Many teaching materials explain one protocol or algorithm at a time. This project combines topology inspection, addressing, routing, ACL security, QoS, fault injection, packet tracing, and network-service simulations in one interface. Its novelty is educational integration and visibility of decision steps, rather than a new routing protocol. The project enables a student to change a scenario and observe the corresponding simulated outcome in a consistent campus model.

3. PROPOSED SYSTEM

3.1 System architecture or topology

The application follows a client–server structure. The React frontend presents dashboards, topology views, simulation controls, and results. The Express/TypeScript backend exposes API routes and invokes separate simulation engines. A SQLite-compatible sql.js layer persists device, link, VLAN, ACL, and experiment data. The logical topology includes an ISP gateway, edge router R1, core routers R2 and R3, distribution/access switching, servers, and end devices. The R1–R2–R3 triangle supplies an alternate path when one core link fails.

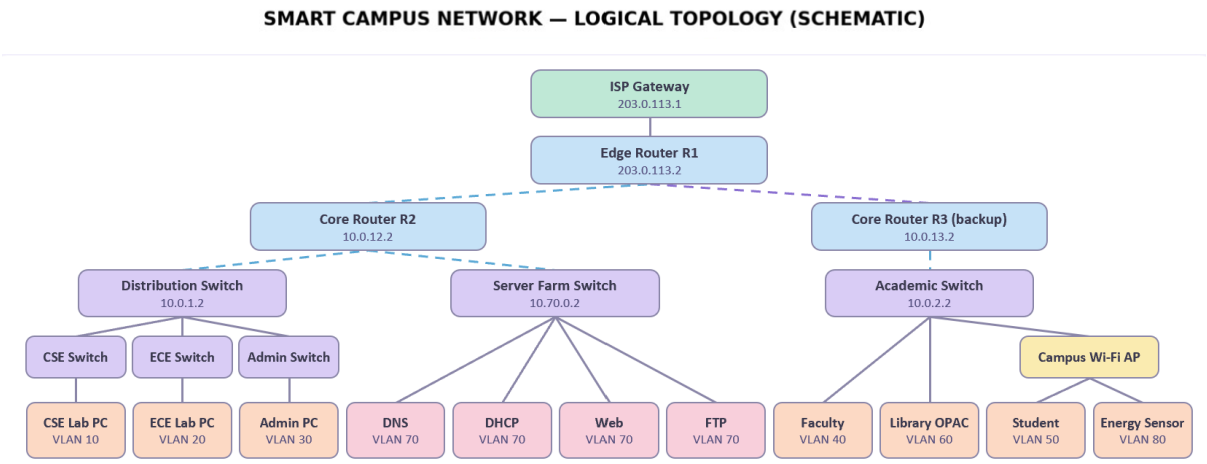


Figure 1. Logical topology schematic based on the project's seeded campus design.

3.2 Baseline mechanism description

The baseline is the normal topology with active links and default routing/security/queue settings. OSPF-style routing chooses a minimum-cost path using Dijkstra's algorithm; RIP-style routing uses a distance-vector calculation and hop-count constraint. The baseline security path evaluates the ordered ACL list. For traffic management, FIFO tail-drop serves packets in arrival order until the buffer is full. These baselines are compared with alternate routing after a failure and strict-priority queueing during congestion.

3.3 Proposed mechanism: algorithm, pseudocode, flowchart

The proposed mechanism is an integrated experiment workflow. The user selects endpoints or a scenario, the backend obtains the current topology, computes the route, checks policy at relevant devices, simulates forwarding, and returns a structured trace. In the fault-tolerance experiment, a link is marked failed, route calculation is repeated, and the post-failure path is compared with the original. In the QoS experiment, the same generated packet sequence is run through both strict-priority and FIFO policies.

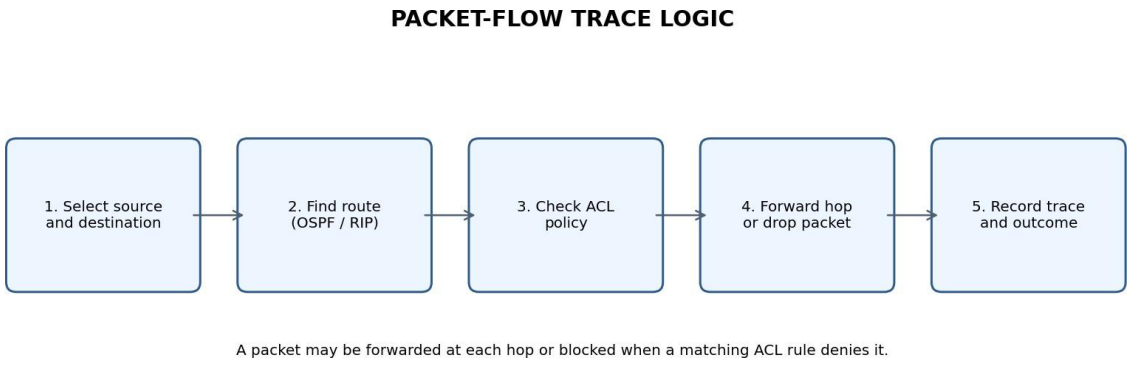


Figure 2. Flowchart: Simplified packet-flow processing sequence.

Pseudocode — controlled link-failure experiment:

```
INPUT: devices, links, source, destination, failed_link, routing_protocol
baseline_path ← FIND_PATH(source, destination, routing_protocol)
modified_links ← links with failed_link.status = FAILED
```

```
failure_path ← FIND_PATH(source, destination, routing_protocol,
modified_links)
IF failure_path is reachable THEN
    estimate convergence time using the simulator's protocol model
    estimate packet loss during the convergence interval
ELSE
    record destination as unreachable and packet loss as 100%
END IF
restore original links; calculate recovery path
RETURN baseline, failure, recovery paths and experiment summary
```

3.4 Novelty statement

The contribution is the unified, interactive comparison of network mechanisms using a seeded campus topology, visible per-hop decisions, controlled failures, and service/protocol teaching modules. The system's novelty is integration and explainability; the underlying routing and queueing algorithms are established techniques.

3.5 Assumptions and design decisions

- Devices and links are represented as structured records, and active/failed status determines graph connectivity.
- Routing and queueing results are generated by software models, not by physical packet transmission.
- The seeded topology and default ACL/VLAN configuration provide reproducible starting conditions.
- Convergence times are synthetic values drawn from configured ranges in the simulator.
- Automated unit tests validate defined scenarios but do not establish real-world throughput or latency.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Table 3. Tools and technologies used in the project

Layer	Technology used	Purpose
Frontend	React 18.3, TypeScript 5.7, Vite 6, Tailwind CSS 3.4 (dark NOC theme)	Interactive dashboard and simulation views
Visualisation	@xyflow/react 12.3, Recharts 2.15, Lucide React 0.468	Topology canvas, charts, icons
Backend	Node.js, Express 4.21, TypeScript 5.7, tsx, cors, dotenv	REST-style API and simulation orchestration
Persistence	sql.js 1.12 (SQLite WebAssembly)	Persist topology and experiment records
Testing	Vitest 2.1.8	Automated unit tests for networking engines
C programs	ANSI C (C99/C11), optional GCC/Clang	Standalone implementations of nine syllabus algorithms

Layer	Technology used	Purpose
Host environment	Node.js v24.x (v18 or higher supported); browser-based frontend	Development and demonstration
Configuration	Backend PORT=5000 (.env); Vite dev server on port 5173 with /api proxy to the backend	Local deployment of the two components

The platform is software-only and does not require physical routers, switches, or a dedicated networking lab. The platform was developed and tested on a Windows host with Node.js v24.x; Node.js v18 or higher and npm v9 or higher are required, and any modern web browser can be used for the interface. The backend API listens on port 5000 and the Vite development server on port 5173, which proxies /api requests to the backend. GCC or Clang is needed only to compile the optional standalone C programs.

4.2 Simulation or testbed setup

The initial dataset defines nine VLANs: 10 (CSE), 20 (ECE), 30 (Administration), 40 (Faculty/Staff), 50 (Student Network), 60 (Central Library), 70 (Data Center Servers), 80 (IoT Sensors), and 99 (Network Management). Each seeded VLAN uses a /24 IPv4 subnet in the 10.x.0.0 range and a corresponding IPv6 /64 prefix. The topology API returns 22 devices and 22 links.

Table 4. Simulation parameters and configured values

Parameter	Configured or documented value	Role in experiment
VLAN count	9 (IDs 10–80 and 99)	Logical department separation
Topology API	22 devices, 22 links	Common graph for experiments
Routing protocols	OSPF-style Dijkstra; RIP-style distance-vector	Path selection comparison
Fault test packets	100 packets per simulated failure experiment	Packet-loss estimate
QoS bottleneck	Default 100 Mbps; queue capacity 60 packets in engine	Congestion and queue comparison
QoS packet generation	Default 12,000 packets/s; maximum 2,000 generated packets	Synthetic offered load
Persistence	SQLite-compatible file: backend/data/campus_network.sqlite	Keep topology and experiment state

4.3 Code structure and explanation of key modules

Table 5. Code structure and responsibilities of key modules

File / module	Responsibility
frontend/src/pages/Dashboard.tsx	Dashboard metrics, traffic visualisation, event feed
frontend/src/pages/TopologyPage.tsx	Interactive network graph and device inspection
backend/src/simulator/graph.ts	Graph representation and active-link neighbours
backend/src/simulator/routing.ts	Dijkstra and distance-vector route calculations
backend/src/simulator/fault_tolerance.ts	Link failure, alternate route, convergence and loss estimates
backend/src/simulator/acl.ts	Ordered first-match packet filtering

File / module	Responsibility
backend/src/simulator/qos.ts	Discrete time-stepped queue simulation with and without QoS
backend/src/simulator/packet_tracer.ts	Per-hop switching, routing and ACL decision trace
backend/src/simulator/addressing.ts	Subnet, VLSM, subnet-membership and IPv6 formatting utilities
backend/src/simulator/protocols.ts	Hamming, CRC, checksum and ARQ/ALOHA-style simulations
backend/src/simulator/services.ts	DHCP, DNS, HTTP, FTP and NAT/PAT educational models
backend/src/database/db.ts and seed.ts	Database initialisation and default network dataset
backend/src/tests/networking.test.ts	Automated tests for addressing, routing, failover, ACL, services, QoS and protocols
c-programs/*.c	Nine standalone C programs: hamming_code, crc, checksum, go_back_n, selective_repeat, aloha, slotted_aloha, csma, csma_cd

The frontend provides eleven pages: Dashboard, Network Topology, VLAN & IP Manager, Routing Simulator, Fault Tolerance Lab, Security Center, Traffic & QoS, Network Services, Protocol Lab, Experiment Reports, and Settings/Docs. Five guided one-click demonstration scenarios are included: a normal web request across VLANs, ACL enforcement, dynamic fault recovery, congestion with QoS, and Hamming (7,4) error correction.

4.4 Challenges faced and how they were solved

- Keeping simulations modular: separate TypeScript modules isolate routing, addressing, security, QoS, services, and protocol logic.
- Showing why a packet was blocked: the packet tracer includes ACL result, matched rule, and a reason in the hop record.
- Demonstrating resilience safely: the fault-tolerance engine modifies a copy of the link list for the experiment, then restores the original topology for recovery analysis.
- Making the project portable: sql.js uses a WebAssembly SQLite engine and avoids a native SQLite build dependency.
- Validating behaviour: 23 Vitest tests cover representative valid and invalid cases, and HTTP/API integration checks and production builds were run for the backend and frontend.

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

Table 6. Performance metrics and justification

Metric	Meaning	Reason
Reachability / path availability	Whether a path exists between endpoints	Measures impact of link failure
Path cost and hop count	Route cost and number of links traversed	Compares route-selection outcomes

Metric	Meaning	Reason
Convergence time	Simulated time for failover route availability	Shows protocol-model difference after failure
Packet loss (%)	Lost packets divided by packets sent × 100	Represents transient impact during failover
Throughput / delay / drop count	QoS simulator's traffic outcomes	Compares queueing policies under load
ACL decision	Allow/deny and matched rule	Validates network segmentation policy
Test pass rate	Passed automated cases divided by total cases	Checks implemented functional scenarios

5.2 Scenarios and parameter values

Table 7. Evaluation scenarios and parameter values

Scenario	Setup	Expected/checked outcome
S1 — Low load / normal routing	All seeded links active; route between online endpoints	Reachable baseline route and routing table
S2 — Medium load / core link failure	Disable link R1–R2; route R1 to a CSE endpoint using OSPF	Alternate route through R3 if one exists
S3 — High load / bottleneck	Run same packet stream through strict-priority and FIFO queue models	Compare throughput, delay and drops by policy
S4 — Security policy	Student VLAN 50 or IoT VLAN 80 to Administration VLAN 30	ACL denies the restricted flow
S5 — No alternate path	Fail an access link to the destination endpoint	Destination becomes unreachable; model reports 100% packet loss

5.3 Results: tables and graphs, baseline vs proposed

The results below were recorded from the running Smart Campus Network application for the fault-tolerance, QoS, and ACL modules. Each scenario was executed once with the default configuration, so the values are reported as single-run simulator outputs and mean ± standard deviation is not computed (see Section 5.5).

Table 8. Verification items and observed results (screenshots in Appendix B, Figures B2–B12)

Verification item	Observed result	Interpretation
Fault tolerance (OSPF): link-r1-r2 severed	46 ms convergence; 1% transient loss; 99/100 packets delivered	Alternate route via R3 maintained connectivity
Fault tolerance (RIP): link-r1-r2 severed	3,557 ms convergence; 53% transient loss; 47/100 packets delivered; route changes from 4 to 5 hops via R3	Connectivity restored, but with far slower convergence and higher loss than OSPF
Routing (OSPF): R1 to CSE Lab PC-01	Cost 18; 4 hops; 4 ms latency	Shortest path R1, R2, SW-Core, SW-CSE, PC-01
Packet tracer: student laptop to web server (TCP/80)	Delivered; 6 hops; 11 ms; permitted by ACL rule #30	Per-hop forwarding and ACL decisions are traceable
ACL: Student VLAN to Administration	Rule #20 DENY; 9 hits; tester verdict DENIED	Restricted student-to-administration traffic matched the deny rule

Verification item	Observed result	Interpretation
ACL: IoT VLAN to Administration	Rule #10 DENY; 0 hits	Rule is enabled; no packets matched it in this run
QoS: Standard FIFO, no QoS	2,000 packets delivered; 0% loss; average delay 66.34 ms	Baseline queueing result for the run
QoS: Strict-priority queueing	2,000 packets delivered; voice loss 0%; voice delay 66.08 ms	Voice traffic protected under the same offered load
Addressing: subnet calculator and VLSM planner	10.10.0.50/24 gives network 10.10.0.0, broadcast 10.10.0.255, 254 hosts; 192.168.1.0/24 split into /25, /26, /27, /28 and /30	Correct network boundaries and hierarchical allocation
Protocol lab: Hamming (7,4)	Data 1011 with error at bit 3; syndrome S = 3	Single-bit error located and corrected
Backend/API and automated tests	23 automated tests passed; 22 HTTP/API checks passed; backend and frontend builds succeeded	All implemented functional scenarios verified

Configured convergence ranges. The fault-tolerance implementation assigns OSPF-style convergence a random value from 30–55 ms and RIP-style convergence from 2,400–3,600 ms when a route remains available. These are simulator assumptions, not measured physical-network timings. The observed OSPF-style (46 ms) and RIP-style (3,557 ms) convergence times fall within these ranges.

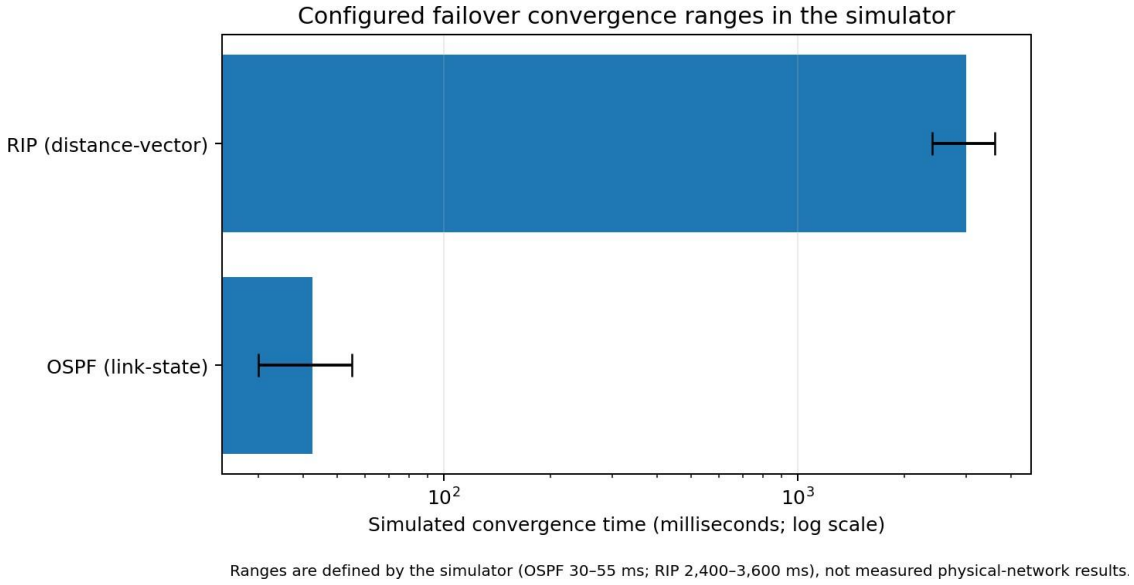


Figure 3. Configured convergence ranges used by the fault-tolerance simulator.

Table 9. Results: baseline (RIP-style routing, FIFO queueing) vs proposed (OSPF-style routing, strict-priority queueing). Single runs; SD not computed, and improvement is calculated from the single-run values

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 (low load)	Hops to CSE Lab PC-01 (normal routing)	4 hops (RIP; single run)	4 hops; cost 18; 4 ms (OSPF; single run)	0

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S2 (medium load)	Convergence time after R1–R2 failure (ms)	3,557 (RIP; single run)	46 (OSPF; single run)	98.7 (reduction)
S2 (medium load)	Transient packet loss (%)	53 (RIP; single run)	1 (OSPF; single run)	98.1 (reduction)
S2 (medium load)	Packets delivered out of 100	47 (RIP; single run)	99 (OSPF; single run)	110.6 (increase)
S3 (high load)	Packets delivered / loss	FIFO: 2,000 delivered; 0% loss (single run)	Strict priority: 2,000 delivered; voice loss 0% (single run)	0 (equal delivered count and loss)
S3 (high load)	Average delay (FIFO) vs voice delay (QoS)	FIFO average: 66.34 ms (single run)	Voice with QoS: 66.08 ms (single run)	n/a (different delay labels; not like-for-like)

5.4 Analysis and discussion

The routing tests establish that the model can find a route to the server farm and generate routing tables. The fault-injection test checks two different conditions: the redundant core should reroute traffic through R3 after the R1–R2 link fails, while failure of the only access path should make the endpoint unreachable. The implementation's convergence ranges make OSPF-style failover much faster than RIP-style failover by design. This supports a comparison of the two simulator models, but the numerical difference should not be presented as a measured result from physical equipment.

The ACL tests demonstrate segmentation: student and IoT traffic directed to Administration is denied, while the configured student web-access rule permits traffic to the campus web server. The QoS engine generates one packet sequence and runs it under two queue policies, which helps isolate the queueing policy as the variable. Exact percentage improvements would require repeated runs across chosen packet rates, packet sizes, durations, bandwidths, and queue capacities, which is left to future work.

5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails

Strengths include a unified interface, visible packet-hop decisions, configurable experiments, modular code, seeded VLANs, and automated regression tests. The simulator can demonstrate route failover and ACL enforcement without risking a live network. Its limits are equally important: the routing model is not a complete OSPF/RIP implementation, convergence is estimated using configured random ranges, traffic is synthetic, and each scenario was executed once, so repeated-run statistics and validation against packet captures are not included. A proposed path fails when no active alternate route exists. Priority queueing can also starve lower-priority traffic under sustained high-priority load; a simplified queue model may not reproduce all effects of real devices.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

Smart Campus Network brings together campus topology visualisation, addressing, routing, failover, ACL security, QoS, packet tracing, service simulation, and protocol labs. The verification suite comprises 23 passing automated tests, 22 passing HTTP/API checks, and successful backend and frontend production builds. The routing tests verify an alternate route through R3 after the tested core-link failure and verify the unreachable case when an access link has no alternate path. The security tests confirm the selected allow/deny examples. These are quantified functional findings; physical throughput or latency improvement is not claimed. In the fault-tolerance experiment, OSPF-style failover converged in 46 ms with 1% transient loss, against 3,557 ms and 53% loss for RIP-style routing.

6.2 Limitations

- The platform is a simulator rather than a real router/switch deployment or packet-level network emulator.
- Convergence values and packet loss are generated by application logic and include randomised ranges.
- Each scenario was executed once, so mean ± SD and confidence intervals are not reported.
- Protocol implementations are educational abstractions and may omit standards-level timers, state machines, and corner cases.
- Standalone C programs require a local compiler to execute separately; the web application does not require them.

6.3 Future enhancements

- Add a repeatable batch-run mode with fixed random seeds and export mean, standard deviation, and confidence intervals.
- Validate selected scenarios using Mininet, ns-3, or packet captures from a controlled testbed.
- Add more traffic patterns, queue disciplines, link delays, and packet-size distributions.
- Expand protocol behaviour and document the exact assumptions for each simulator.
- Export experiment data to CSV/JSON and generate plots automatically for each user-selected scenario.

7. REFERENCES

References (IEEE style)

[1] J. Moy, “OSPF Version 2,” RFC 2328, Internet Engineering Task Force, Apr. 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2328

[2] G. Malkin, “RIP Version 2,” RFC 2453, Internet Engineering Task Force, Nov. 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2453

[3] S. Deering and R. Hinden, “Internet Protocol, Version 6 (IPv6) Specification,” RFC 2460, Internet Engineering Task Force, Dec. 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2460

[4] F. Baker, “Requirements for IP Version 4 Routers,” RFC 1812, Internet Engineering Task Force, Jun. 1995. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1812

[5] R. Braden, Ed., “Requirements for Internet Hosts—Communication Layers,” RFC 1122, Internet Engineering Task Force, Oct. 1989. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1122

[6] J. Postel, “User Datagram Protocol,” RFC 768, Internet Engineering Task Force, Aug. 1980. [Online]. Available: https://www.rfc-editor.org/rfc/rfc768

[7] A. Bhargava and P. Saxena, “Smart Campus Network: source code and documentation,” Vellore Institute of Technology, Vellore, India, Oct. 2026.

Appendix A: Important source code or configuration listings

A.1 Build, test and run commands

```
# Install dependencies (from the project root)
npm install
cd backend && npm install
cd ../frontend && npm install
cd ..
# Run the automated tests
cd backend && npm test
# Production builds (backend tsc, frontend tsc + vite build)
npm run build
# Start the backend on port 5000 (terminal 1)
cd backend && npm start
# Start the frontend on port 5173 (terminal 2)
cd frontend && npm run dev
# Compile a standalone C program (example)
cd c-programs && gcc -Wall -O2 hamming_code.c -o hamming_code
```

A.2 Configuration listings

Backend environment file (.env.example)

```
PORT=5000
NODE_ENV=development
```

Backend package.json (scripts and dependencies)

```
"scripts": { "dev": "tsx watch src/index.ts", "build": "tsc",
             "start": "node dist/index.js", "test": "vitest run" },
"dependencies": { "cors": "^2.8.5", "dotenv": "^16.4.7",
                  "express": "^4.21.2", "sql.js": "^1.12.0" },
"devDependencies": { "tsx": "^4.19.2", "typescript": "^5.7.2",
                     "vitest": "^2.1.8", ... }
```

Backend tsconfig.json (compiler options)

```
"target": "ES2022", "module": "NodeNext", "moduleResolution": "NodeNext",
"outDir": "./dist", "rootDir": "./src", "strict": true,
"esModuleInterop": true, "skipLibCheck": true, "resolveJsonModule": true
```

Frontend package.json (scripts and dependencies)

```
"scripts": { "dev": "vite", "build": "tsc && vite build",
             "preview": "vite preview" },
"dependencies": { "@xyflow/react": "^12.3.6", "react": "^18.3.1",
                  "react-dom": "^18.3.1", "recharts": "^2.15.0",
                  "lucide-react": "^0.468.0", "clsx": "^2.1.1",
                  "tailwind-merge": "^2.5.5" },
"devDependencies": { "vite": "^6.0.3", "typescript": "^5.7.2",
                     "tailwindcss": "^3.4.16", "postcss": "^8.4.49", ... }
```

Frontend vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173, host: true,
    proxy: { '/api': { target: 'http://localhost:5000', changeOrigin: true
} },
  },
});
```

Frontend tailwind.config.js (dark NOC theme)

```
darkMode: 'class',
theme: { extend: {
  colors: { noc: { bg: "#090D16", card: "#0F172A", hover: "#1E293B",
    border: "#1E293B", text: "#F8FAFC", muted: "#94A3B8", cyan: "#06B6D4",
    blue: "#3B82F6", emerald: "#10B981", amber: "#F59E0B",
    rose: "#F43F5E", purple: "#8B5CF6" } },
  fontFamily: { mono: ['JetBrains Mono', 'Fira Code', 'Consolas',
'monospace'],
                sans: ['Inter', 'system-ui', 'sans-serif'] } } }
```

A.3 Example ACL test case

The automated tests verify that TCP traffic from 10.50.0.105 (Student VLAN 50) to 10.30.0.25 (Administration VLAN 30) is denied by rule acl-20. They also verify that IoT traffic from 10.80.0.15 to the same administration endpoint is denied by acl-10, and that student TCP web traffic to 10.70.0.12 is permitted by acl-30.

A.4 Example fault-tolerance test

The test calls runFailureExperiment() for router-r1 to pc-cse-1 while failing link-r1-r2. The test expects the pre-failure route to use router-r2 and the during-failure route to include router-r3. A separate test fails the access link link-swcse-pccse1 and expects the destination to be unreachable with 100% simulated packet loss.

Appendix B: Additional screenshots, packet captures, raw result tables

This appendix collects the supporting packet-flow diagram, raw result table and application screenshots referenced in the report.

B.1 Packet-flow logic diagram

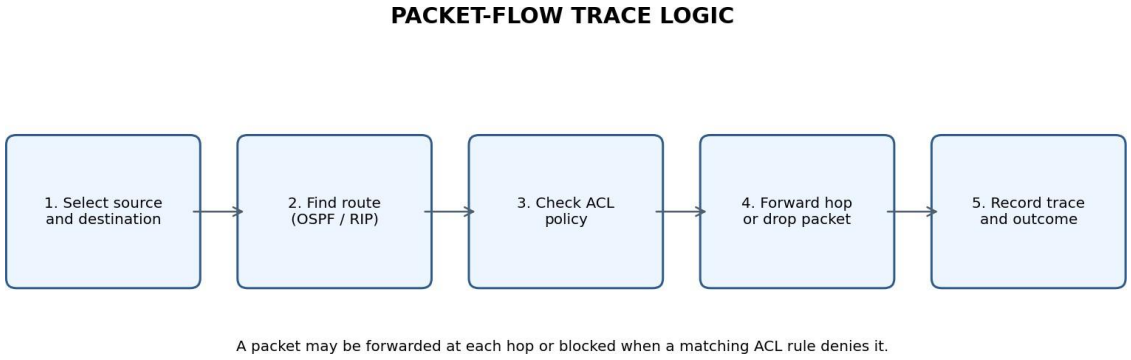


Figure B1. Logic diagram generated from the packet-tracing workflow; not a packet capture.

B.2 Raw result table

Table 10. Raw results from the running application (single runs)

Run ID	Scenario / parameters	Baseline metric	Proposed metric	Notes
Run 1	Sever link-r1-r2; RIP vs OSPF; 100 test packets	RIP: 3,557 ms convergence; 53% loss; 47/100 delivered	OSPF: 46 ms convergence; 1% loss; 99/100 delivered	Single run each
Run 2	QoS screen; 2,000 packets	FIFO: 2,000 delivered; 0% loss; 66.34 ms average delay	Strict priority: 2,000 delivered; 0% voice loss; 66.08 ms voice delay	Single run
Run 3	ACL rules enabled	Rule #20 student-to-admin DENY: 9 hits	Rule #10 IoT-to-admin DENY: 0 hits	Hit counts are rule counters, not packet-loss values

B.3 Screenshots of the running application

The following screenshots were captured from the running Smart Campus Network application (localhost:5173). The values are simulator outputs, not measurements from physical campus networking hardware.

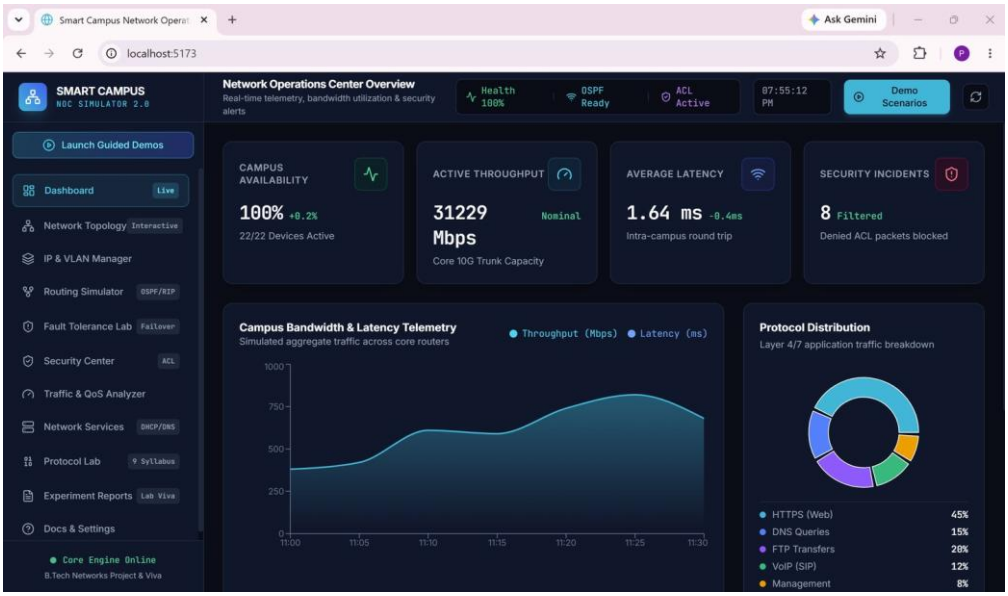


Figure B2. Dashboard: campus availability (22/22 devices active), throughput, latency, security incidents, bandwidth telemetry and protocol distribution.

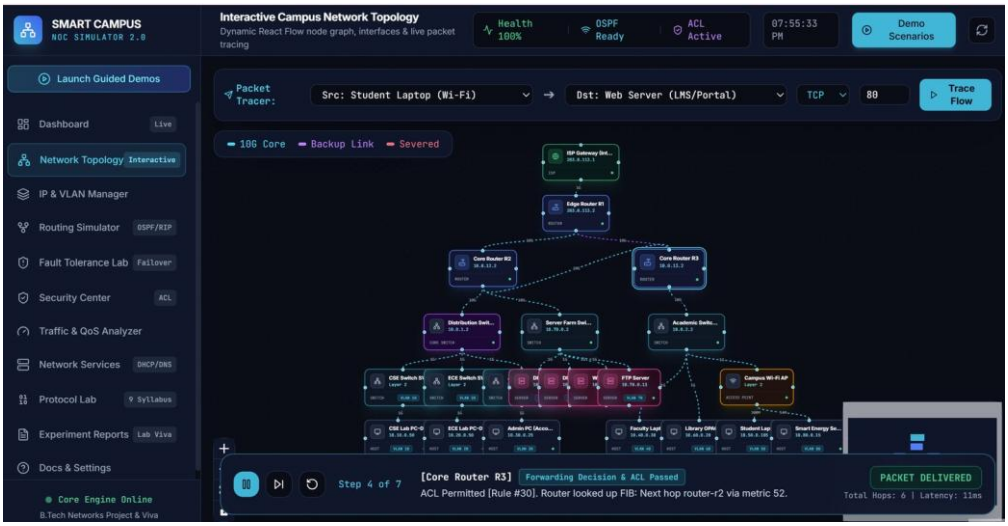


Figure B3. Network topology with the packet tracer: a TCP/80 request from a student laptop to the LMS web server is delivered in 6 hops (11 ms), with an ACL check at each router.

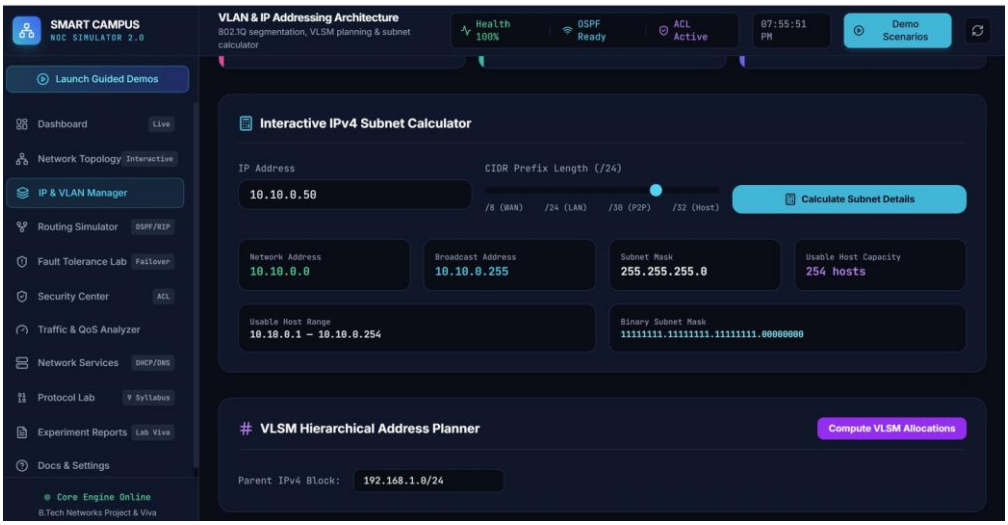


Figure B4. IPv4 subnet calculator: 10.10.0.50/24 gives network 10.10.0.0, broadcast 10.10.0.255 and 254 usable hosts.

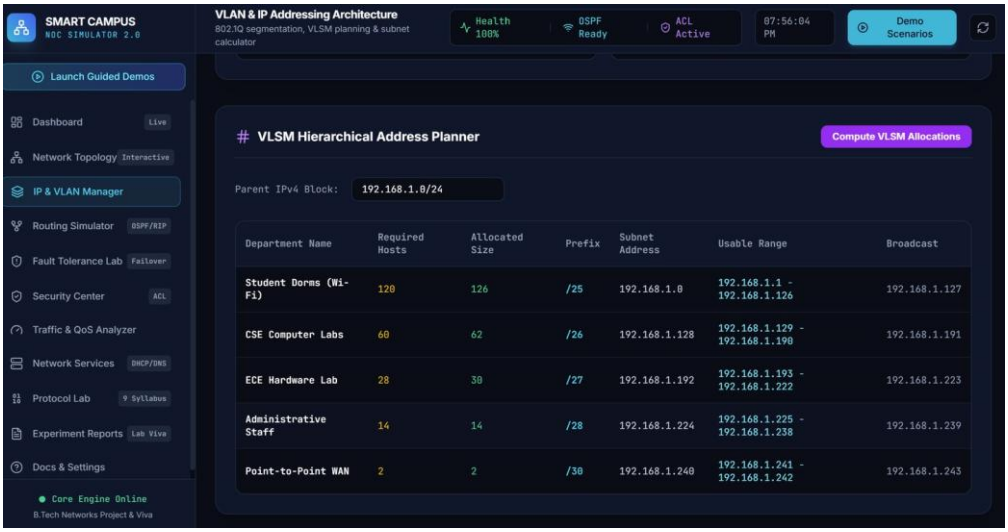


Figure B5. VLSM planner: 192.168.1.0/24 divided into /25, /26, /27, /28 and /30 subnets for five departments.

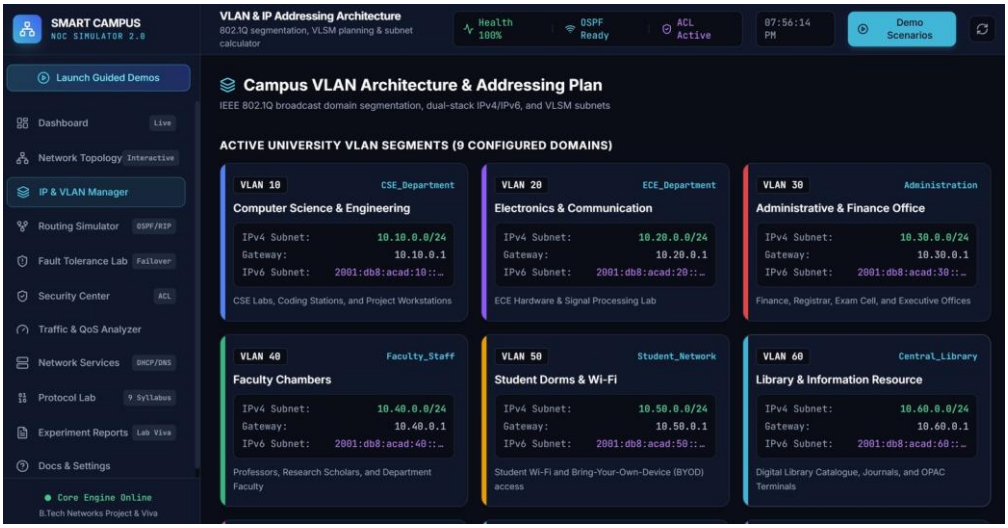


Figure B6. VLAN addressing plan: nine configured VLANs with IPv4 subnets, gateways and IPv6 prefixes.

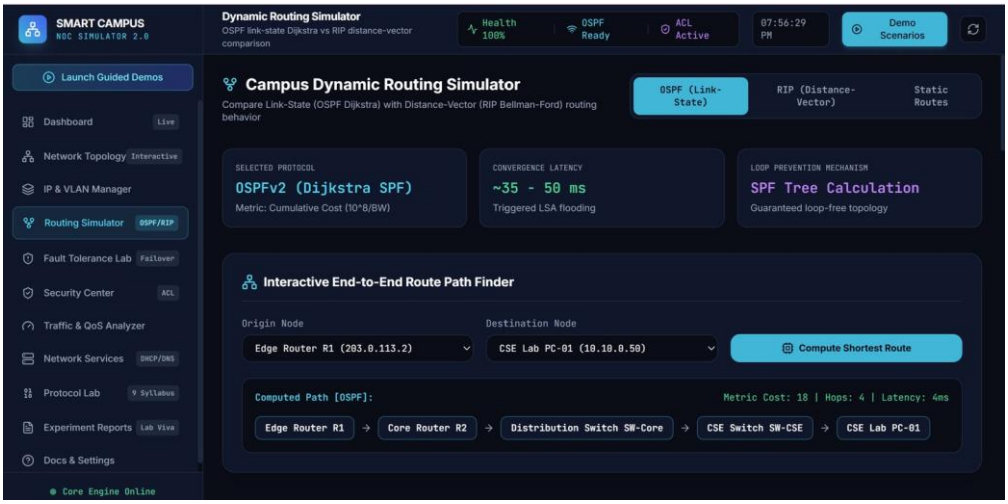


Figure B7. Routing simulator (OSPF): shortest path from Edge Router R1 to CSE Lab PC-01 with cost 18, 4 hops and 4 ms latency.

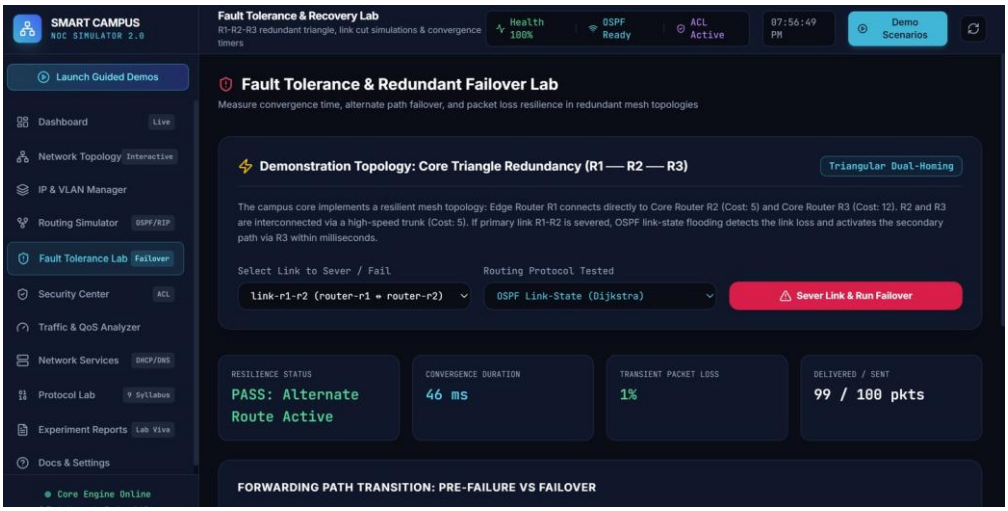


Figure B8. Fault tolerance lab (OSPF): after link-r1-r2 is severed, the alternate route stays active with 46 ms convergence, 1% loss and 99/100 packets delivered.

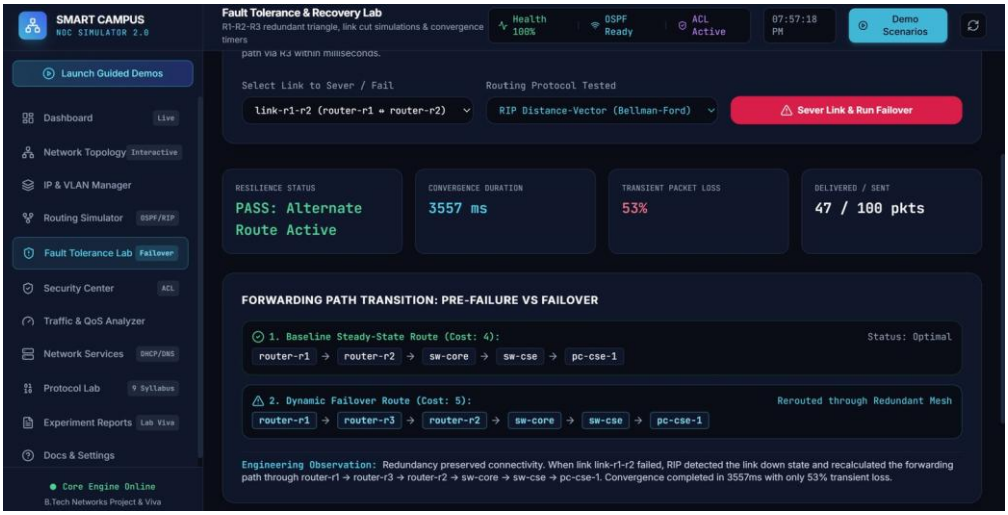


Figure B9. Fault tolerance lab (RIP): the same failure converges in 3,557 ms with 53% loss (47/100 delivered); the route grows from 4 to 5 hops via R3.

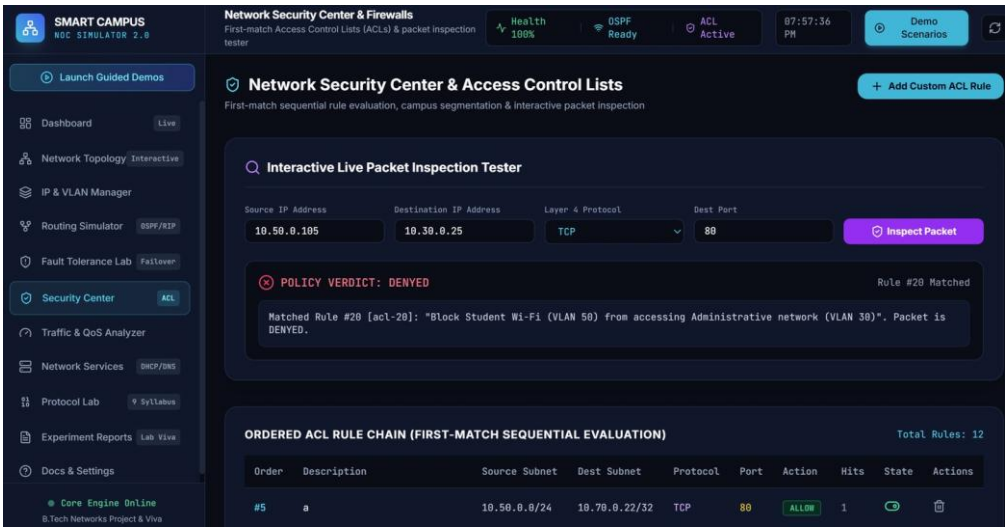


Figure B10. Security Center packet tester: TCP/80 from 10.50.0.105 (Student VLAN 50) to 10.30.0.25 (Administration VLAN 30) is denied by ACL rule #20.



Figure B11. Ordered ACL rule chain (12 rules) with hit counters, including deny rules #10 (IoT) and #20 (student) for the Administration subnet.

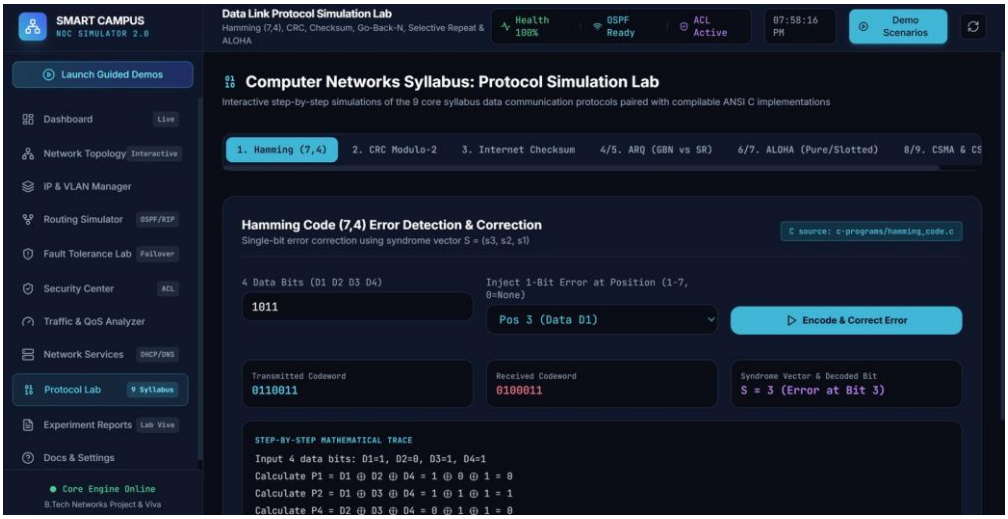


Figure B12. Protocol lab: Hamming (7,4) encoding of 1011 with a single-bit error at position 3, located by syndrome S = 3 and corrected.

B.4 Note on statistical reporting

Each scenario was executed once, so mean ± standard deviation and percentage improvements based on repeated runs are not reported. A fully quantitative evaluation would repeat each scenario at least five times under identical settings and report the mean and sample standard deviation; this is listed as future work.